

Использование языка программирования Julia для решения математических задач

Часть I

Авторы:

Чуракова Юлия Романовна

*Студентка факультета математики и компьютерных наук
Кубанского Государственного Университета*

Алексеев Евгений Ростиславович

*Кандидат технических наук, доцент
Доцент кафедры информационных технологий
Кубанского Государственного Университета*

Полная версия пособия доступна в репозитории:

<https://github.com/JuliaChurakova/Julia>

Литература

1. Энгхейм, Э. Julia в качестве второго языка / Э. Энгхейм. – Москва: ДМК Пресс, 2023. – 446 с.
2. The Julia Programming Language: сайт. – URL: <https://julialang.org> (дата обращения: 10.10.2025).
3. Белов, Г. В. Краткое описание языка программирования Julia и некоторые примеры его использования/ Г. В. Белов. – Москва: МГТУ им. Н.Э. Баумана, 2024. – 108 с.
4. Шеррингтон, М. Осваиваем язык Julia / М. Шеррингтон. – Москва: ДМК Пресс, 2017. – 416 с.
5. Антонюк, В. А. Язык Julia как инструмент исследователя / В. А. Антонюк. – Москва: Физический факультет МГУ им. М. В. Ломоносова, 2019. – 48 с.

Глава 8 Инструменты графического представления результатов

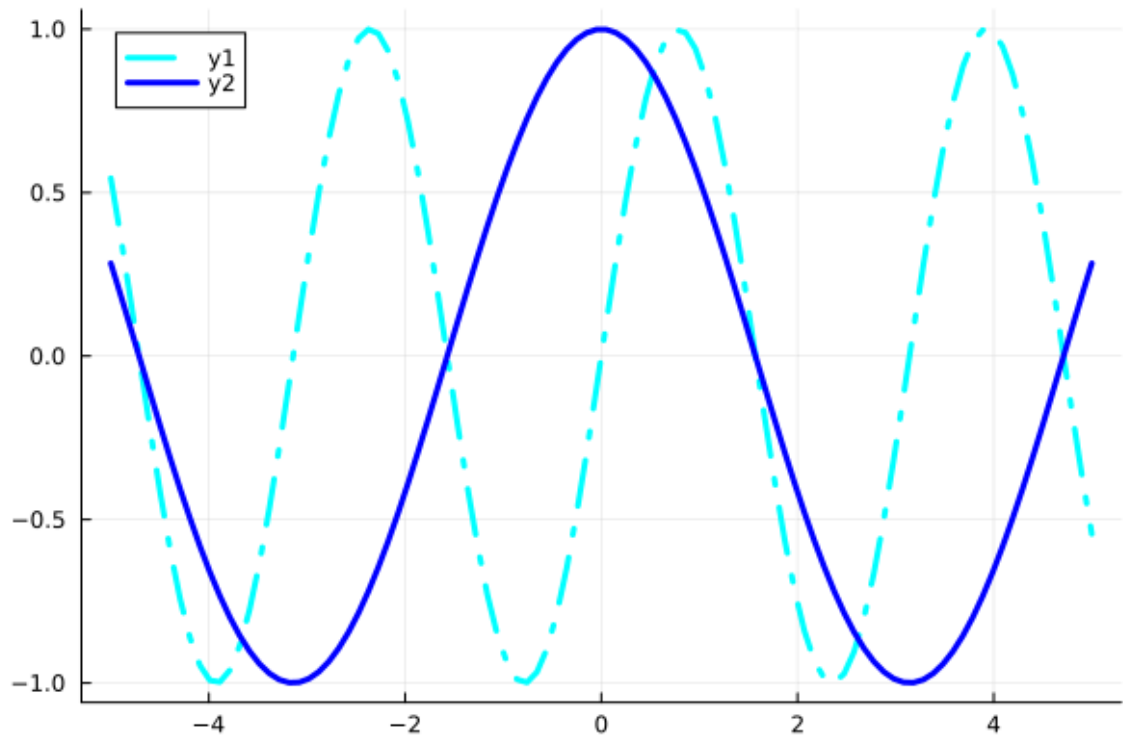
В Julia для визуализации данных часто используют пакет Plots. Этот пакет предоставляет единый интерфейс для множества различных бэкендов визуализации, таких как PyPlot (для работы с Matplotlib), GR (для работы с Gnuplot), PGFPlotsX и других.

Для получения дополнительной информации о пакете см. [документацию](#).

Вот несколько простых примеров использования пакета Plots для создания различных типов графиков.

```
In [1]: using Plots
x = range(-5, stop = 5, length = 100)
y = sin.(2 .* x)
z = cos.(x)
plot(x, y, color=:cyan, width=3, linestyle = :dashdot)
plot!(x, z, color=:blue, linestyle = :solid, width=3)
```

Out[1]:



Для построения графика функция `plot` имеет вид **`plot(x,y,[type])`** `x` – массив абсцисс, `y` – массив ординат, параметр `type` является строкой, в которой определяется цвет графика и тип линии, в строку `type` можно включать следующие параметры:

- **стиль линии** – строка, определяющая стиль линии;

Атрибут	Стиль линии
<code>dashdot</code>	Штрихпунктирная линия
<code>dash</code>	Штриховая линия
<code>dot</code>	Пунктирная линия
<code>solid</code>	Жирная линия

- **цвет** – строка, определяющая цвет линии;

Атрибут	Цвет
<code>red</code>	Красный
<code>green</code>	Зелёный
<code>blue</code>	Синий
<code>cyan</code>	Голубой
<code>purple</code>	Пурпурный
<code>yellow</code>	Жёлтый
<code>black</code>	Чёрный

Атрибут	Цвет
white	Белый

- **width** – строка, определяющая толщину линии.

Кроме просмотра графиков на экране можно сохранить их в файл. В Plots это делается очень просто с помощью команд.

Сохранить график в файл в формате .png

```
savefig("myplot.png")
```

Или так

```
png("myplot.png")
```

Сохранить график в файл в формате .pdf

```
savefig("myplot.pdf")
```

Или так

```
pdf("myplot.png")
```

Если нужно построить на одном графике несколько функций, это можно сделать с использованием вызова функций с восклицательным знаком.

Функция **scatter**, используемая в библиотеке Plots, позволяет отображать данные в виде точек на плоскости.

```
scatter(x, y; [type])
```

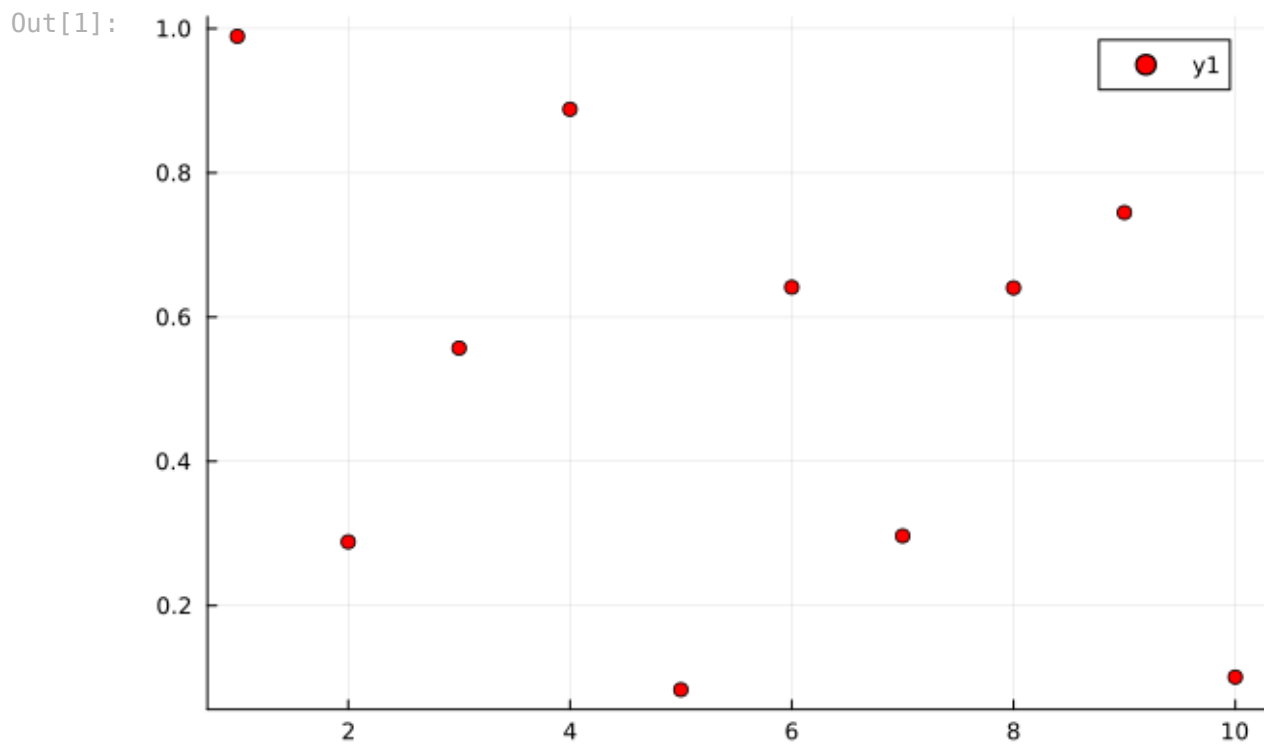
где x – массив абсцисс, y – массив ординат, type – дополнительные параметры (например, цвет точек, размер, маркеры и т. д.).

In [1]: **using** Plots

```
x = 1:10
```

```
y = rand(10)
```

```
scatter(x, y, color=:red)
```



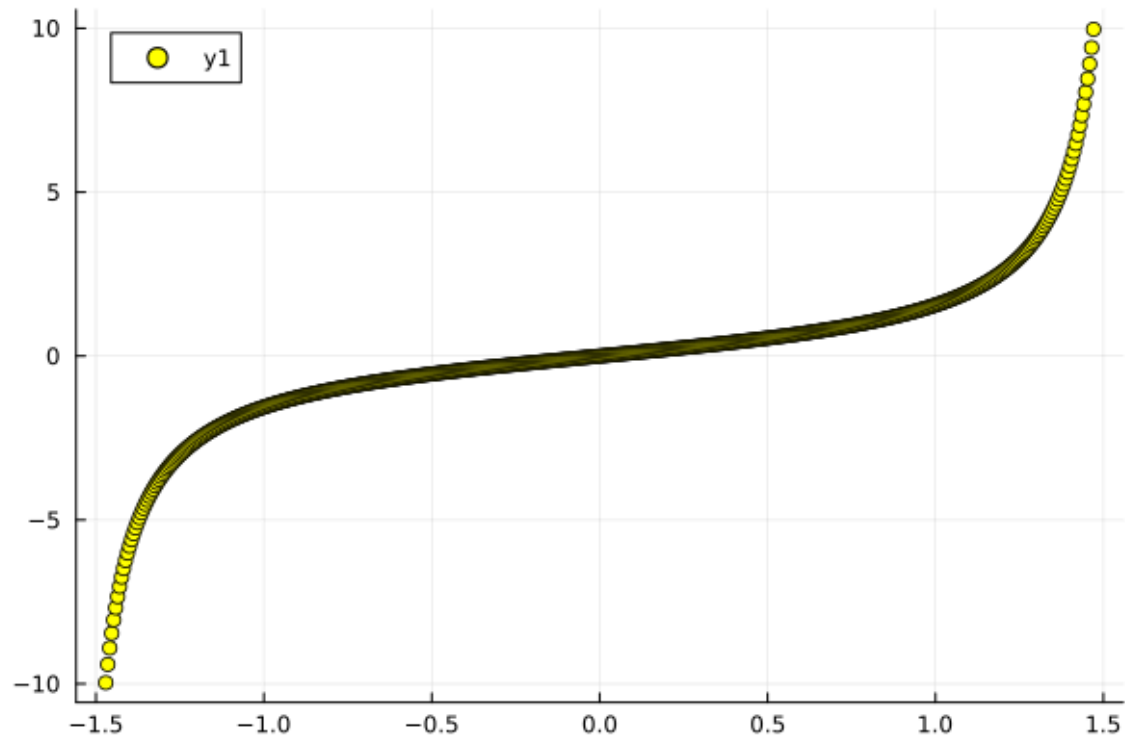
```
In [2]: using Plots

# Определяем диапазон значений для x
x = LinRange(-π/2 + 0.1, π/2 - 0.1, 500)

y = tan.(x)

scatter(x, y, color=:yellow)
```

Out[2]:

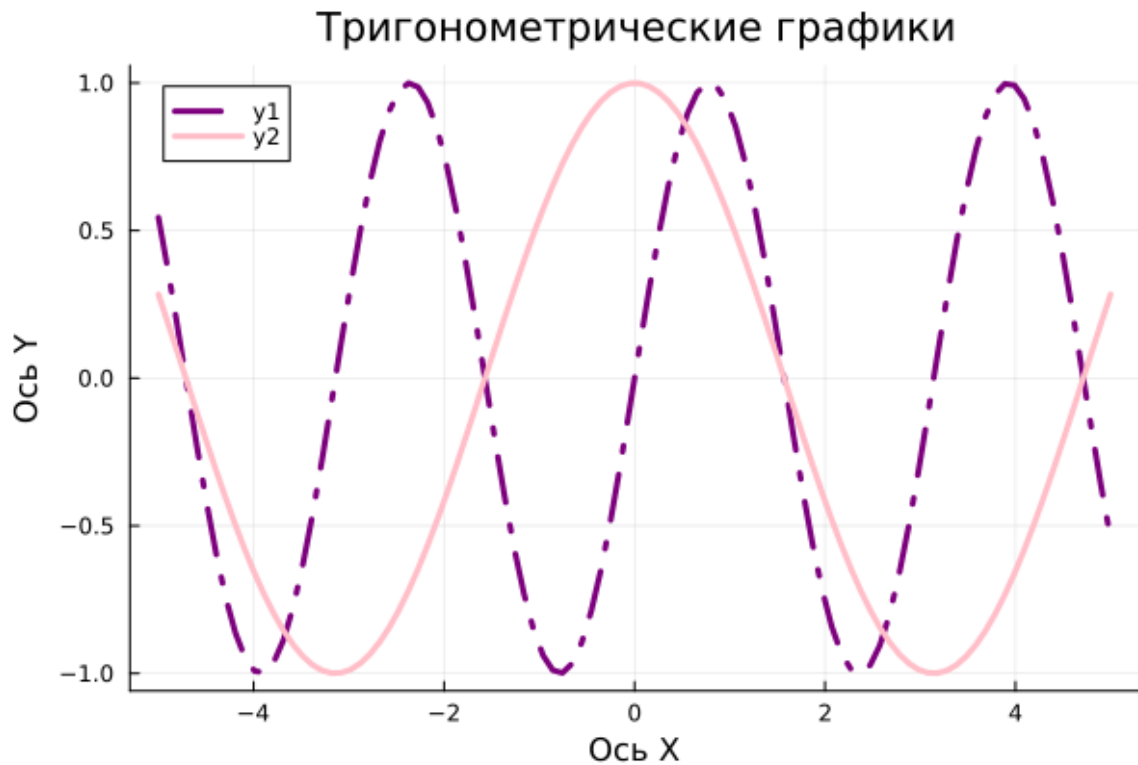


8.1 Заголовок, подписи, сетка, легенда

Заголовок и подписи к осям ставятся командами **title**, **xlabel** и **ylabel**, принимающими единственный аргумент – строку.

```
In [3]: x = range(-5, stop = 5, length = 100)
y = sin.(2 .* x)
z = cos.(x)
plot(x, y, color=:purple, width=3, linestyle = :dashdot, title = "Тригономет",
plot!(x, z, color=:pink, linestyle = :solid, width=3)
```

Out[3]:



Часто необходимо, чтобы в подписях на осях или легенде содержались верхние или нижние индексы, греческие буквы, различные значки и прочие математические символы. Для этого Plots имеет режим совместимости с командами LaTeX. В строке внутри пары символов $\$$ можно вводить обычные формулы LaTeX ($\$$ – начало и конец формулы)

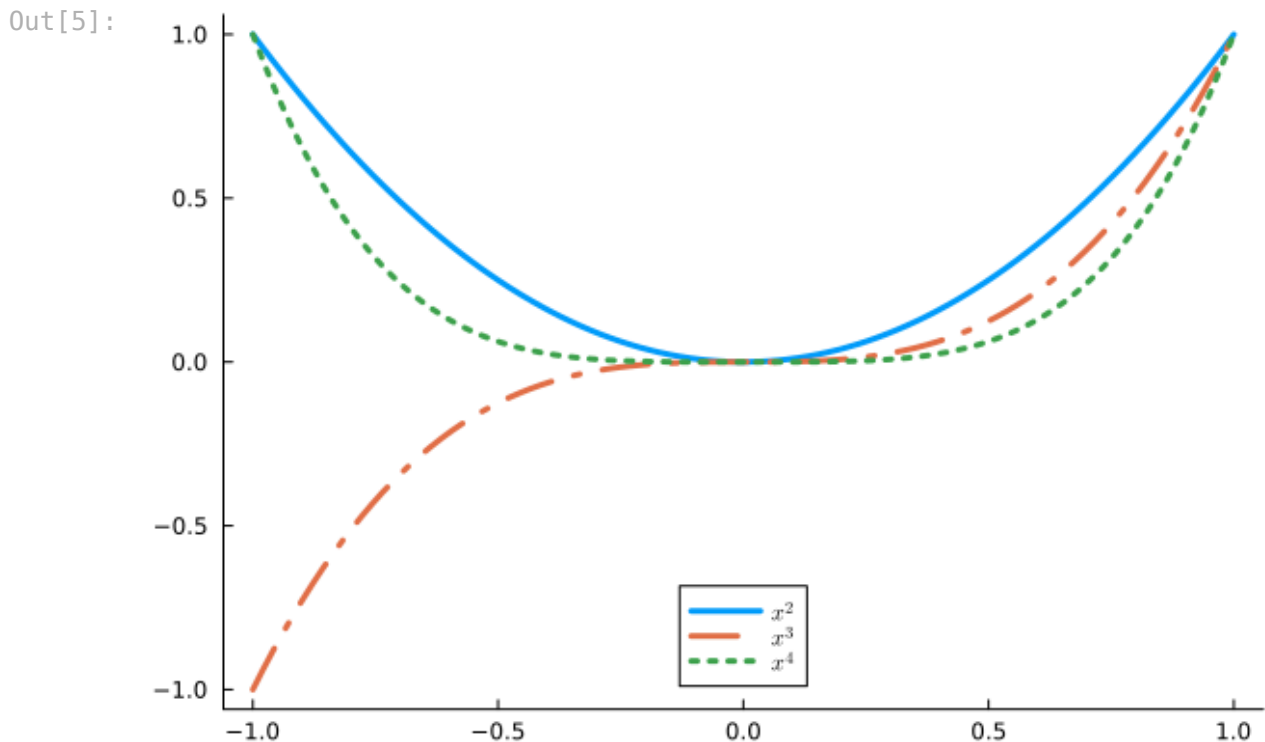
Вывод легенды осуществить командой **legend()**. Чтобы переместить легенду или изменить её расположение, можно использовать:

Параметры	Строка
best	наилучший
topright	сверху справа
topleft	сверху слева
bottomleft	снизу слева
bottomright	снизу справа
left	по центру слева
right	по центру справа
bottom	снизу по центру
top	сверху по центру
inside	по центру

Чтобы отключить легенду, можно использовать **legend=false**.

Ещё один часто встречающийся элемент графиков – сетка. Для получения сетки служит функция **grid**, у которой единственный логический аргумент (True – сетка будет, False – нет).

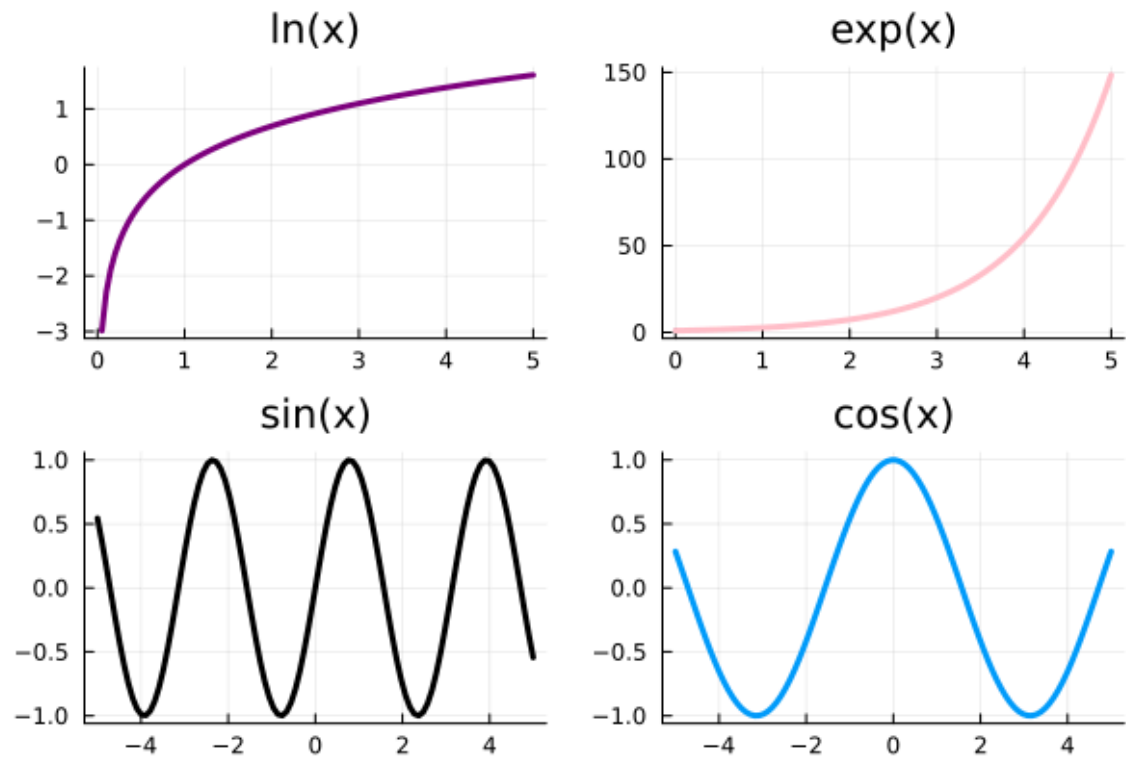
```
In [5]: t = range(-1, stop = 1, length = 100)
x = t.^2
y = t.^3
z = t.^4
plot(t, x, label = "\$x^2\$", legend=:bottom, grid=:false, width=3)
plot!(t, y, label = "\$x ^3\$", linestyle = :dashdot, width=3)
plot!(t, z, label = "\$x ^4\$", linestyle = :dot, width=3)
```



8.2 Несколько графиков на одном полотне

```
In [6]: x = range(0, stop = 5, length = 100)
y=log.(x)
p1 = plot(x,y, title="ln(x)", color=:purple, width=3, legend=false)
x = range(0, stop = 5, length = 100)
y=exp.(x)
p2 = plot(x,y, title="exp(x)", color=:pink, width=3, legend=false)
x = range(-5, stop = 5, length = 100)
y = sin.(2 .* x)
p3 = plot(x,y, title="sin(x)", color=:black, width=3, legend=false)
x = range(-5, stop = 5, length = 100)
y = cos.(x)
p4 = plot(x,y, title="cos(x)", width=3, legend=false)
plot(p1, p2, p3, p4)
```

Out[6]:



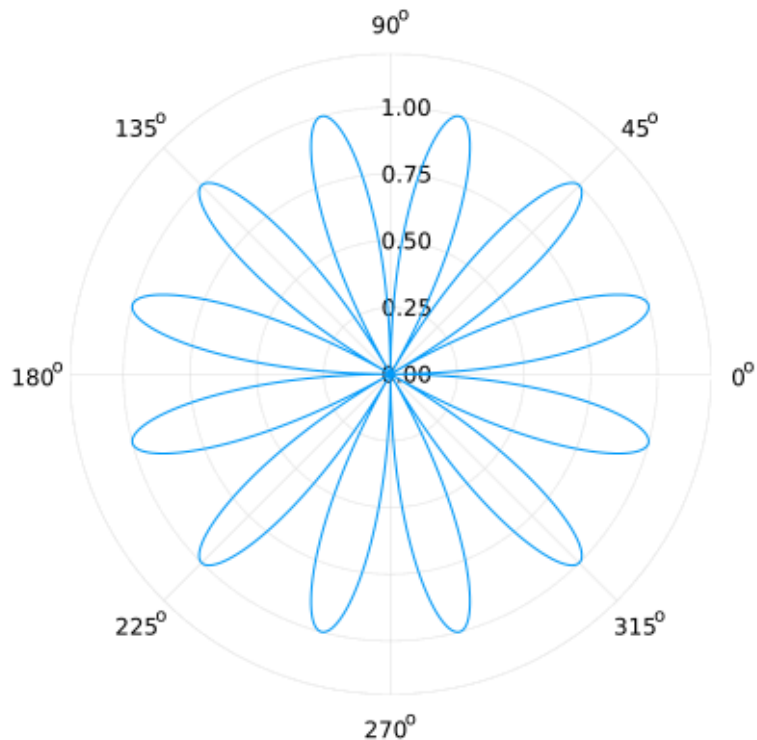
8.3. Графики в полярных координатах

Для построения графиков в полярных координатах нужно при вызове функции `plot()` указать `proj=:polar`.

Пример. Построить график следующей зависимости: $r(t)=\sin(6t)$, $t \in [0; 2\pi]$ в полярных координатах.

```
In [7]: t=range(0,2pi, 1000)
rt=sin.(6.0.*t)
plot(t,rt,proj=:polar, legend=false)
```

Out[7]:



8.4 Трехмерные графики в Plots

Для построения трехмерных графиков в Julia с использованием библиотеки Plots можно использовать функцию **surface**, которую необходимо использовать внутри функции **plot**.

```
plot(surface(x, y, z))
```

Пример 1. Построить график функции:

$$z(x, y) = \sqrt{x^2 + y^2}$$

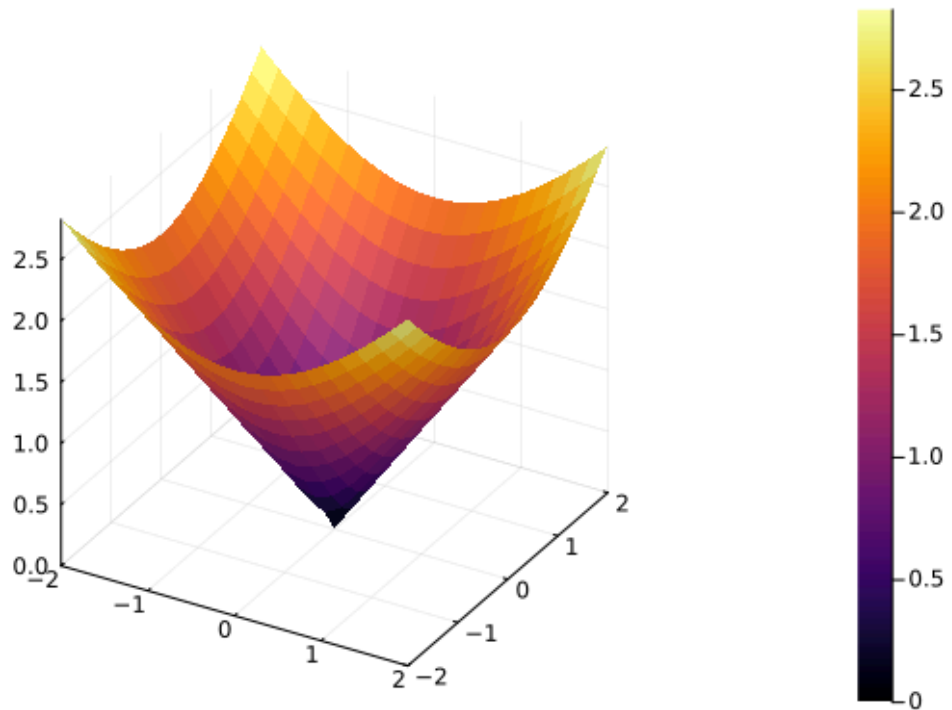
```
In [4]: using Plots

# Создаем сетку значений x и y
x = -2:0.2:2
y = -2:0.2:2

# Создаем матрицы X, Y для сетки
X = [xi for xi in x, yi in y]
Y = [yi for xi in x, yi in y]

# Вычисляем значения Z = sqrt(x^2 + y^2)
Z = [sqrt(xi^2 + yi^2) for (xi, yi) in zip(X, Y)] #функция zip позволяет пара
# Строим 3D график
plot(surface(X, Y, Z))
```

Out[4]:



Пример 2. Построить поверхность однополостного гиперболоида, уравнение которого задано в параметрическом виде:

$$x(u, v) = ch(u)\cos(v),$$

$$y(u, v) = ch(u)\sin(v),$$

$$z(u, v) = sh(u).$$

In [9]: **using** Plots

```
# Шаг для параметров
```

```
h = π / 50
```

```
# Формируем вектор u и v
```

```
u = 0:h:π
```

```
v = 0:2h:2π
```

```
x = [cosh(ui) * cos(vi) for ui in u, vi in v]
```

```
y = [cosh(ui) * sin(vi) for ui in u, vi in v]
```

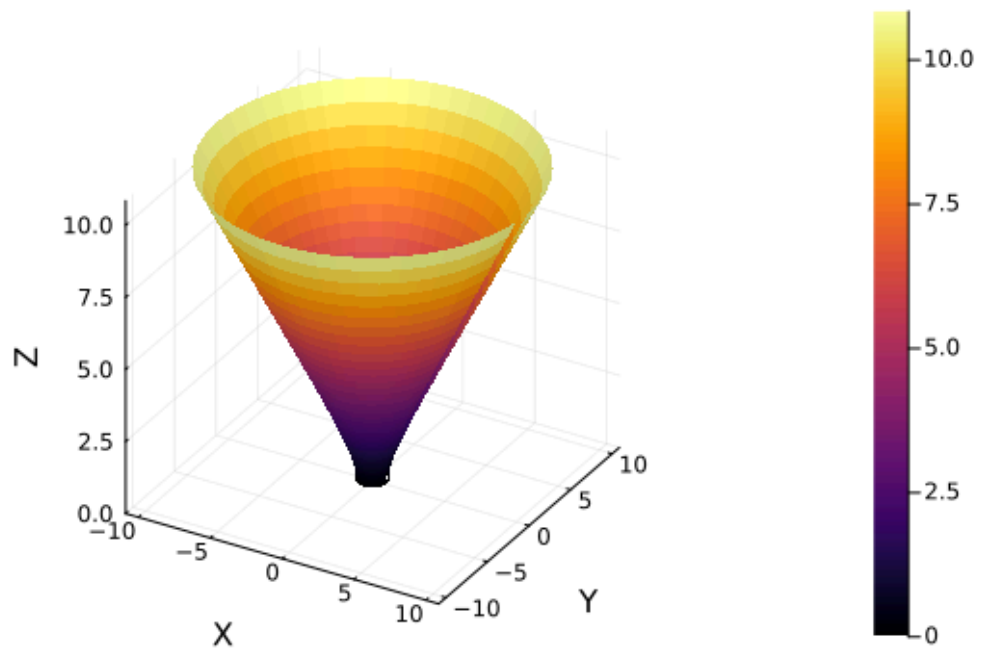
```
z = [sinh(ui) for ui in u, vi in v]
```

```
# Строим 3D-график поверхности
```

```
plot(surface(x, y, z), xlabel="X", ylabel="Y", zlabel="Z", title="ГРАФИК ОДН
```

Out[9]:

ГРАФИК ОДНОПОЛОСТНОГО ГИПЕРБОЛОИДА



Пример 3. Построить поверхность сферы, уравнение которой задано в параметрическом виде:

$$x(u, v) = \sin(u)\cos(v),$$

$$y(u, v) = \sin(u)\sin(v),$$

$$z(u, v) = \cos(u).$$

In [10]: **using** Plots

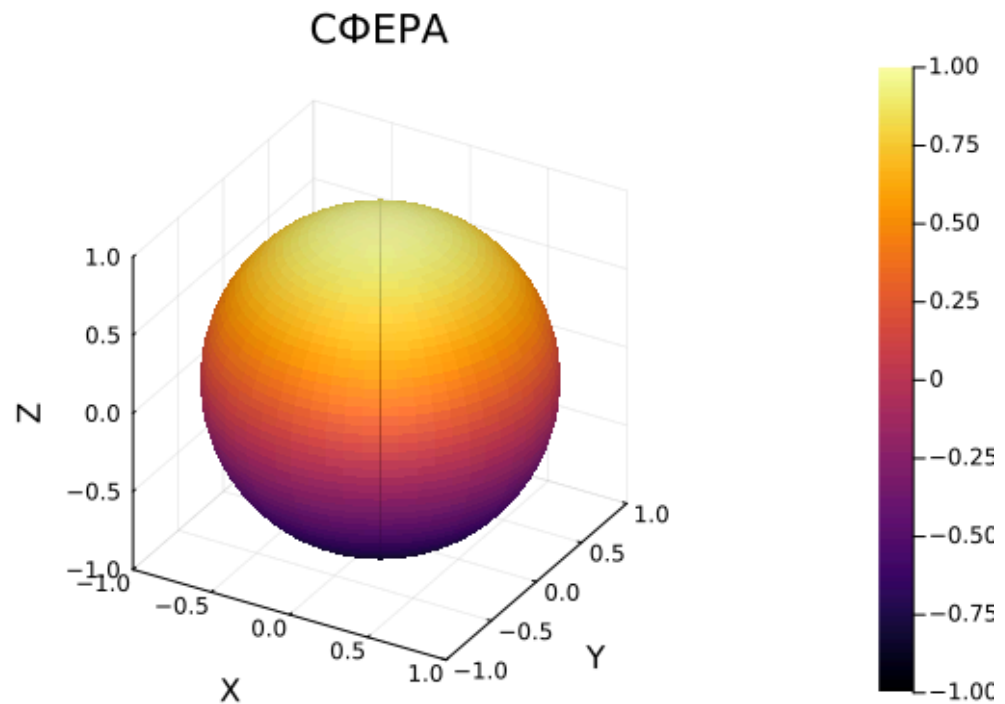
```
# Шаг для параметров
h = π / 60

# Формируем вектор u и v
u = 0:h:π
v = 0:2h:2π

x = [sin(ui) * cos(vi) for ui in u, vi in v]
y = [sin(ui) * sin(vi) for ui in u, vi in v]
z = [cos(ui) for ui in u, vi in v]

# Строим 3D-график поверхности
plot(surface(x, y, z), xlabel="X", ylabel="Y", zlabel="Z", title="СФЕРА")
```

Out[10]:



8.5 Библиотека PyPlot и ее использование с Plots

Для визуализации данных в Julia также существует библиотека PyPlot, которая предоставляет интерфейс для работы с популярной Python-библиотекой matplotlib.

Чтобы использовать **Plots** и **PyPlot** в одном коде, важно помнить, что обе эти библиотеки предоставляют функции с одинаковыми именами. Чтобы избежать конфликта имен, Julia предоставляет возможность явно указывать, из какой библиотеки или модуля должна быть вызвана конкретная функция. Для этого достаточно написать имя библиотеки перед функцией, разделив их точкой. Например, если вы хотите использовать функцию `plot` из библиотеки `Plots`, пишите **`Plots.plot()`**, а для **`PyPlot`** – **`PyPlot.plot()`**.

```
In [6]: using Plots    # Подключаем библиотеку Plots
        using PyPlot   # Подключаем библиотеку PyPlot

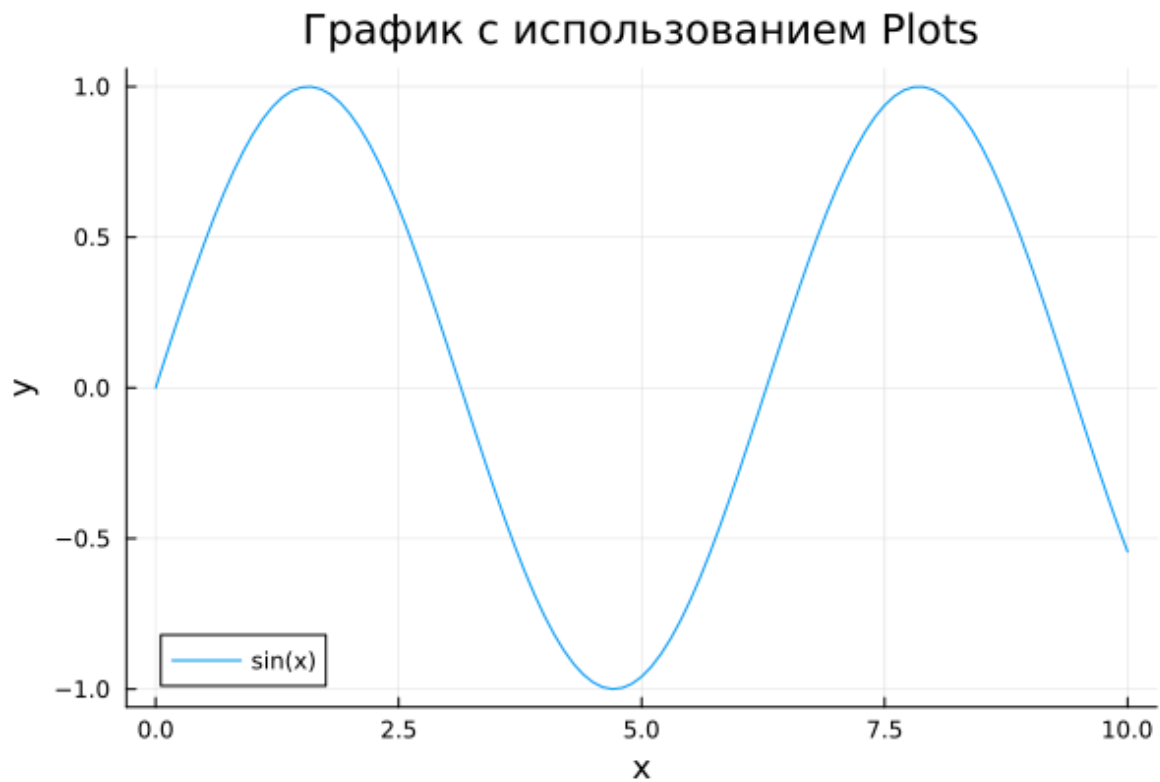
        x = 0:0.1:10
        y = sin.(x)

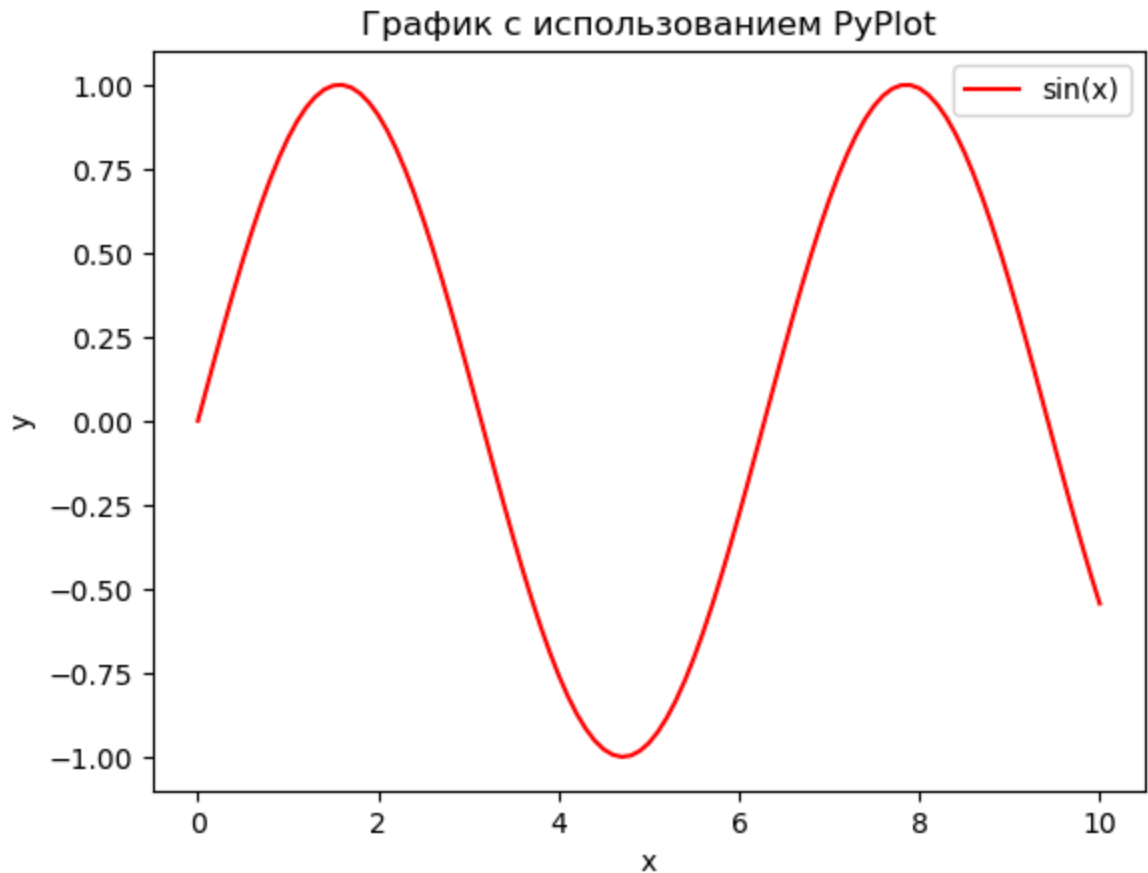
        # Строим график с использованием Plots.plot
        p1 = Plots.plot(x, y, label="sin(x)", title="График с использованием Plots",

        # Строим график с использованием PyPlot.plot
        p2 = PyPlot.plot(x, y, label="sin(x)", color="red") # Явно указываем исполь
```

```
PyPlot.xlabel("x")
PyPlot.ylabel("y")
PyPlot.title("График с использованием PyPlot")
PyPlot.legend()

# Отображаем оба графика
display(p1) # Показываем график, построенный с использованием Plots
PyPlot.show() # Показываем график, построенный с использованием PyPlot
```





In [7]: **using** PyPlot

```
#График с одной линией
x = collect(0:0.1:10)
y = sin.(x) # Применяем sin к каждому элементу массива x

PyPlot.figure() # Создаем новый график
PyPlot.plot(x, y, label="sin(x)", color="blue", linestyle="--", marker="o")
PyPlot.xlabel("x") # Подписываем ось X
PyPlot.ylabel("sin(x)") # Подписываем ось Y
PyPlot.title("Линейный график функции sin(x)") # Заголовок графика
PyPlot.legend() # Легенда
PyPlot.show() # Показываем график

#График с несколькими линиями
y2 = cos.(x) # Применяем cos к каждому элементу x

PyPlot.figure() # Создаем новый график
PyPlot.plot(x, y, label="sin(x)", color="blue")
PyPlot.plot(x, y2, label="cos(x)", color="red")
PyPlot.xlabel("x")
PyPlot.ylabel("y")
PyPlot.title("График sin(x) и cos(x)")
PyPlot.legend()
PyPlot.show()

#Гистограмма
data = randn(1000) # Генерация случайных данных
```

```
PyPlot.figure() # Создаем новый график
PyPlot.hist(data, bins=30, edgecolor="black")
PyPlot.xlabel("Значения")
PyPlot.ylabel("Частота")
PyPlot.title("Гистограмма случайных данных")
PyPlot.show()
```

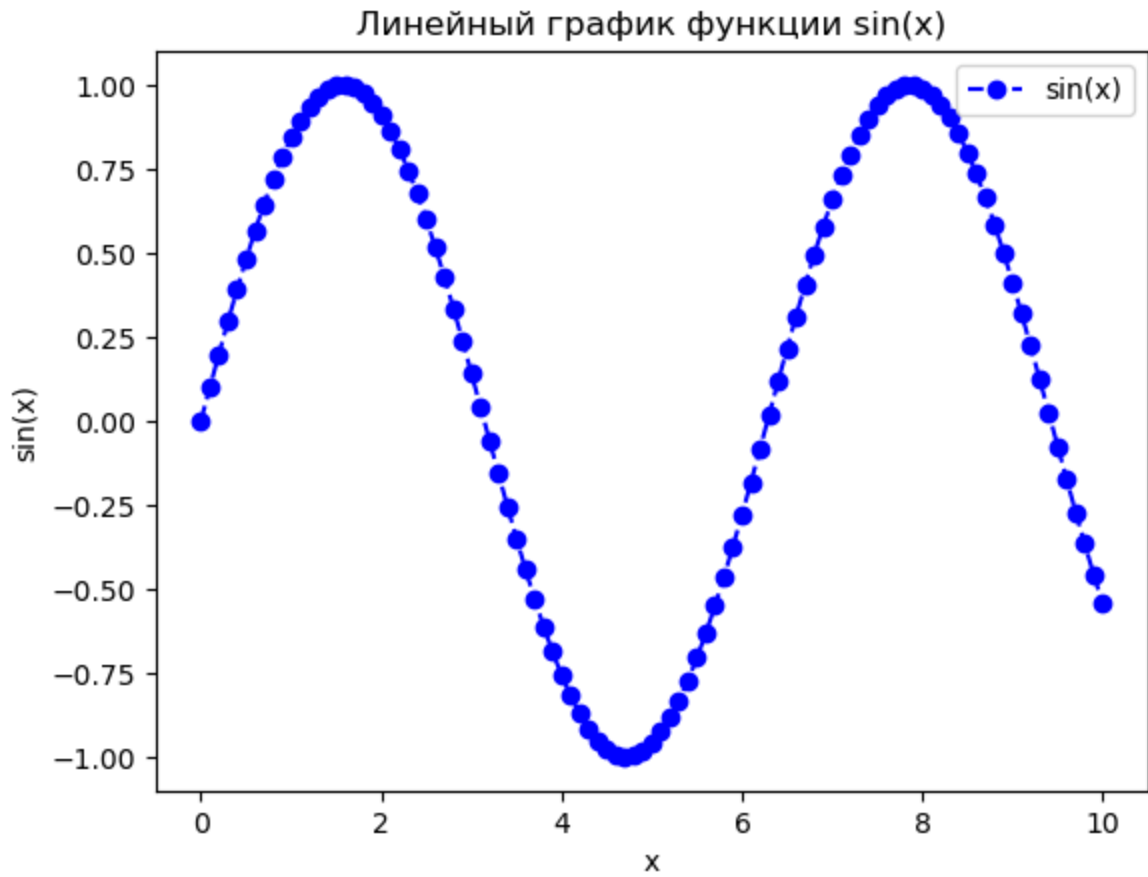
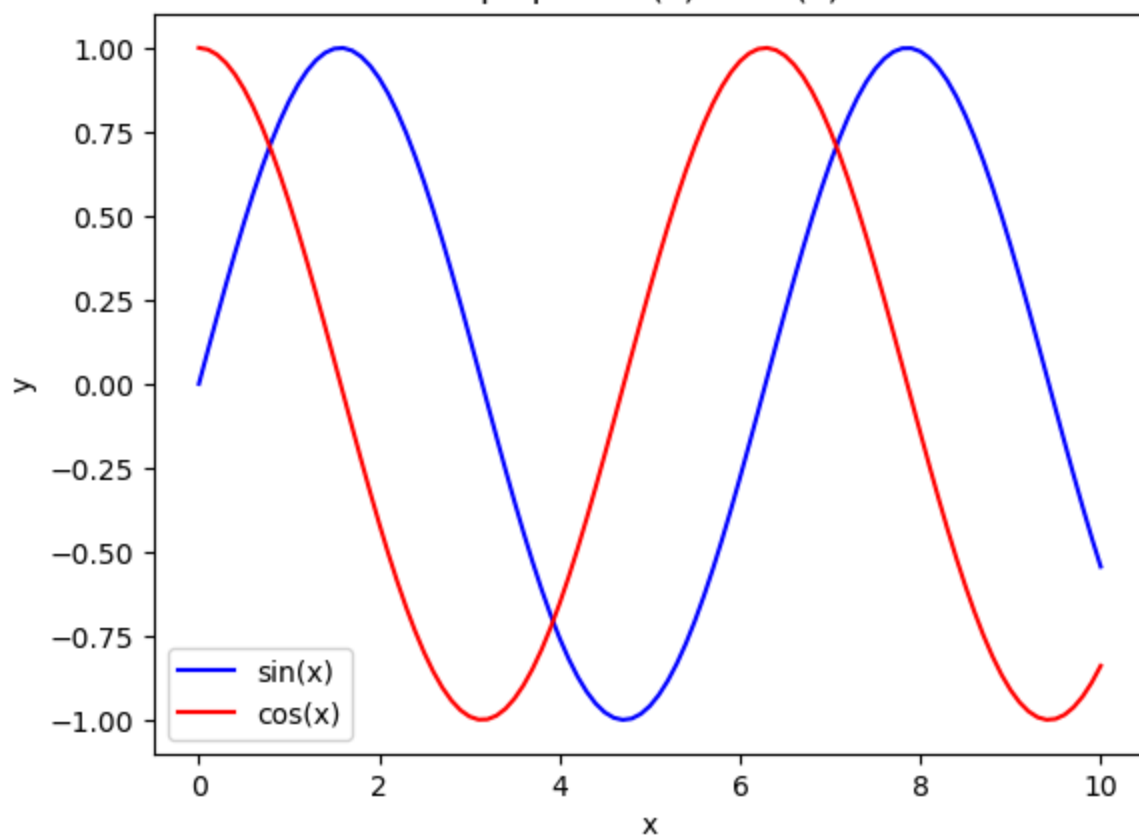
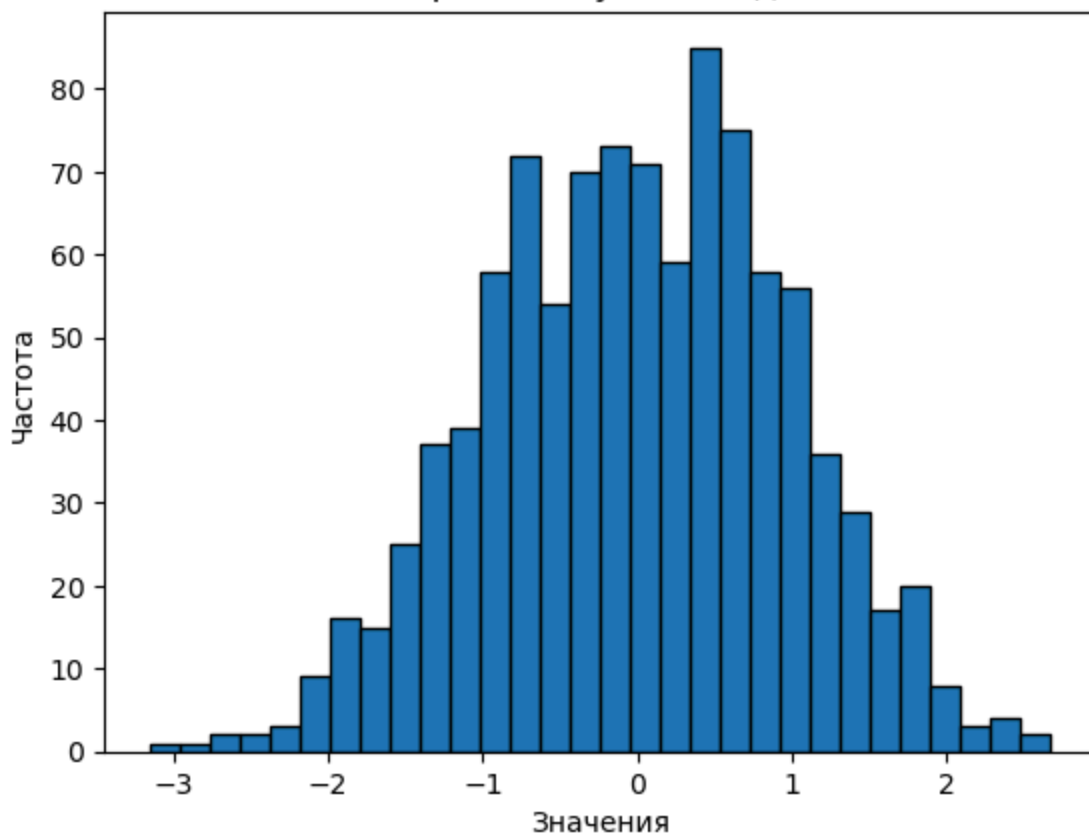


График $\sin(x)$ и $\cos(x)$



Гистограмма случайных данных



В Julia для построения 3D-графиков с использованием библиотеки PyPlot используется функция **surf(x, y, z)**

Пример 4. Построить график функции $z(x, y) = \pm\sqrt{x^2 + y^2}$

In [8]: **using** PyPlot

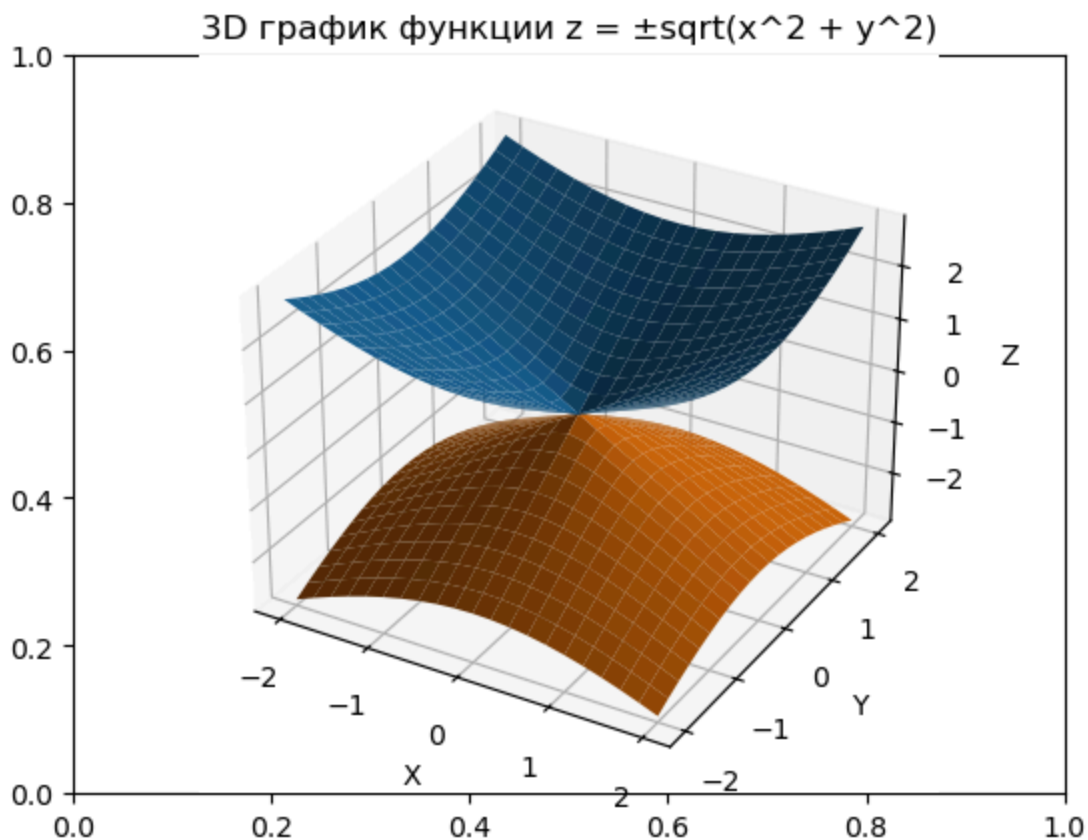
```
# Генерация данных для x и y
x = -2:0.2:2
y = -2:0.2:2

# Создаем матрицы X, Y для сетки
X = [xi for xi in x, yi in y]
Y = [yi for xi in x, yi in y]

# Вычисляем значения Z и Z1 для функции z(x, y) = ±(sqrt(x^2 + y^2))
Z = [sqrt(xi^2 + yi^2) for (xi, yi) in zip(X, Y)] # Положительная часть
Z1 = [-sqrt(xi^2 + yi^2) for (xi, yi) in zip(X, Y)] # Отрицательная часть

# Строим 3D поверхности для положительной и отрицательной части
PyPlot.surf(X, Y, Z) # Положительная часть
PyPlot.surf(X, Y, Z1) # Отрицательная часть

# Подписываем оси и добавляем заголовок
PyPlot.xlabel("X")
PyPlot.ylabel("Y")
PyPlot.zlabel("Z")
PyPlot.title("3D график функции z = ±sqrt(x^2 + y^2)")
```



```
Out[8]: PyObject Text(0.5, 1.0, '3D график функции  $z = \pm\sqrt{x^2 + y^2}$ ')
```